

Đặc trưng và data association (Features and Data Association)

PHỤ THUỘC A/01-hình-học-da-thi (kiểm định hình học là bước cuối của mọi pipeline khớp điểm); A/02-mô-hình-camera (gỡ méo trước khi dùng hình học); A/06-bat-dinh-va-lan-truyen-sai-so (σ của phép đo pixel chính là thứ chương này quyết định, và nó là trọng số cho mọi thứ ở back-end).

TÓM TẮT

Chương này về bài toán mà phần còn lại của cây coi như đã giải xong: *pixel nào ở khung này là cùng một điểm vật lý với pixel nào ở khung kia?* Đây là bài toán *tổ hợp* — có lời giải đúng hoặc sai — khác hẳn bài toán ước lượng liên tục ở back-end, và khác biệt ấy quyết định mọi công cụ dùng để giải nó. Nội dung có bốn phần. Một: cách tìm điểm đáng theo dõi, từ tiêu chí Harris/Shi-Tomasi tới detector học được, và tiêu chí thật để chọn giữa chúng. Hai: cách khớp chúng — mô tả rồi so khớp, hay theo dõi trực tiếp bằng optical flow — và vì sao hai cách phù hợp với hai chế độ hoạt động khác nhau. Ba: hai thứ bị coi nhẹ nhất mà ảnh hưởng lớn nhất, là *phân bố không gian* của đặc trưng và *độ chính xác subpixel*; cái thứ nhất quyết định điều kiện số của Hessian, cái thứ hai quyết định trực tiếp σ tức trọng số của mọi phần dư. Bốn: loại bỏ outlier bằng RANSAC và họ của nó, với phân tích vì sao cỡ mẫu tối thiểu quan trọng hơn người ta nghĩ. Nếu chỉ nhớ một điều: *front-end quyết định hệ thống hỏng theo kiểu nào, back-end chỉ quyết định nó chính xác tới đâu* — và trong sản phẩm, kiểu hỏng quan trọng hơn.

KIẾN THỨC CẦN CÓ TRƯỚC

- **đặc trưng (feature)** — một vùng ảnh nhỏ có tính chất đủ đặc biệt để nhận ra lại ở ảnh khác. Gồm hai phần tách rời: *detector* tìm ra nó ở đâu, *descriptor* mô tả nó trông thế nào.
- **bài toán cửa sổ (aperture problem)** — nhìn qua một lỗ nhỏ vào một cạnh thẳng, ta chỉ biết cạnh dịch chuyển vuông góc với nó bao nhiêu, không biết nó trượt dọc bao nhiêu. Đây là lý do góc theo dõi được còn cạnh thì không, và nó là gốc của mọi tiêu chí detector.
- **ma trận cấu trúc (structure tensor)** — ma trận $M = \sum_w \nabla I \nabla I^T$ lấy trên một cửa sổ. Hai giá trị riêng của nó nói vùng đó là vùng phẳng (cả hai nhỏ), cạnh (một lớn một nhỏ), hay góc (cả hai lớn).
- **optical flow** — trường vectơ mô tả chuyển động biểu kiến của các mẫu cường độ giữa hai ảnh. KLT theo dõi một mảng nhỏ bằng cách giải trực tiếp cho độ dịch làm cường độ khớp nhất.
- **ratio test** — quy tắc của Lowe: chỉ chấp nhận khớp nếu khoảng cách tới ứng viên gần nhất nhỏ hơn đáng kể so với ứng viên gần thứ hai. Nó loại các đặc trưng lặp lại, thứ mà khoảng cách tuyệt đối không loại được.
- **RANSAC** — lập: lấy mẫu tối thiểu, khớp mô hình, đếm inlier, giữ mô hình tốt nhất. Nó tìm *tập nhất quán lớn nhất*, không phải nghiệm chính xác nhất — nên nó luôn phải theo sau bởi một bước tinh chỉnh.
- **tinh chỉnh subpixel** — ước lượng vị trí đặc trưng chính xác hơn một pixel, thường bằng cách khớp parabol vào hàm điểm số quanh cực đại. Nó quyết định trực tiếp σ của phép đo.

CÂU HỎI MỞ ĐẦU

- Q1. Thuật toán năm điểm cần năm mẫu, tám điểm cần tám. Với 50% outlier, chênh lệch số vòng lặp RANSAC giữa hai lựa chọn ấy là bao nhiêu — và vì sao con số đó lớn hơn nhiều so với trực giác “chỉ hơn ba điểm”?
- Q2. Hai hệ thống trích cùng số lượng đặc trưng với cùng chất lượng khớp. Một hệ chính xác hơn hẳn. Nếu một khác biệt về front-end có thể giải thích điều đó mà *không* liên quan gì tới detector hay descriptor.
- Q3. Tinh chỉnh subpixel cải thiện σ từ 0,5 px xuống 0,15 px. Vì sao con số đó ảnh hưởng tới *mọi* thứ ở back-end, và nó ảnh hưởng thế nào tới hiệp phương sai đầu ra?
- Q4. SuperPoint và LightGlue cho nhiều khớp đúng hơn ORB một cách rõ rệt trong điều kiện khó. Vì sao nhiều hệ sản phẩm vẫn dùng ORB, và câu trả lời “vì GPU đắt” là chưa đủ?
- Q5. Hành lang văn phòng có hàng chục cánh cửa giống hệt nhau. Front-end khớp điểm giữa hai cánh cửa khác nhau một cách hoàn hảo — khoảng cách descriptor rất nhỏ. Vì sao ratio test không cứu được ở đây, và thứ gì mới cứu được?

1.1 Đặt vấn đề

Phần A xây dựng toàn bộ bộ máy ước lượng trên một giả thiết mà nó không bao giờ kiểm: rằng ta biết pixel nào tương ứng với pixel nào. A/01 mục 7 nêu nó là giả thiết ngầm thứ tư; chương này là chỗ trả nợ.

Bài toán gốc: cho hai ảnh của cùng một cảnh chụp từ hai chỗ khác nhau, tìm các cặp pixel là ảnh của cùng một điểm vật lý. Nghe đơn giản, và nó khó vì bốn lý do chồng lên nhau. *Thứ nhất, không gian tìm kiếm lớn*: một ảnh 640×480 có ba trăm nghìn pixel, nên ghép cặp vét cạn là chín mươi tỉ phép so sánh. *Thứ hai, phần lớn pixel không phân biệt được*: một pixel giữa bức tường trắng giống hết hàng nghìn pixel khác. *Thứ ba, vẻ ngoài thay đổi*: cùng một điểm nhìn từ góc khác, dưới ánh sáng khác, ở tỉ lệ khác, trông không giống nhau. *Thứ tư, và đây là lý do nghiêm trọng nhất, có những thứ trông giống nhau mà không phải cùng một điểm* — cửa sổ trên một toà nhà, viên gạch trên sàn, cánh cửa trong hành lang.

Ba lý do đầu là lý do *khó*; lý do thứ tư là lý do *nguy hiểm*, và nó khác về bản chất. Với ba lý do đầu, thất bại nghĩa là không tìm được khớp, và hệ thống biết mình không tìm được. Với lý do thứ tư, thất bại nghĩa là tìm được một khớp *sai* trông rất thuyết phục, và hệ thống không biết. Đây là **perceptual aliasing**, và nó là kẻ thù số một của SLAM trong môi trường nhân tạo — chính là môi trường mà robot sản phẩm hoạt động.

Ai gặp bài toán này? Mọi ứng dụng thị giác hình học, và giải pháp được phát triển chủ yếu trong cộng đồng thị giác máy tính chứ không phải robot học. Lịch sử có ba giai đoạn đáng biết vì mỗi giai đoạn để lại một bài học. Giai đoạn một, trước SIFT: theo dõi mảng ảnh trực tiếp (KLT), hoạt động tốt khi chuyển động nhỏ và hỏng hoàn toàn khi lớn. Giai đoạn hai, SIFT [1] và con cháu: mô tả bất biến với tỉ lệ và xoay, cho phép khớp giữa hai ảnh bất kỳ; đây là bước nhảy làm SfM quy mô lớn khả thi. Giai đoạn ba, từ khoảng 2018: detector và matcher học được, cải thiện rõ rệt ở điều kiện khó.

Cách tiếp cận hiển nhiên — và cách gần như ai cũng nghĩ tới đầu tiên — là làm descriptor *tốt hơn* cho tới khi không còn khớp sai. Cách đó không bao giờ thành công, và lý do đáng nói rõ vì nó định hình toàn bộ kiến trúc: *không có descriptor nào phân biệt được hai cánh cửa thật sự giống hệt nhau*, vì thông tin phân biệt không nằm trong vùng ảnh cục bộ. Nó nằm ở *ngữ cảnh* — cánh cửa này ở đâu so với các thứ khác — và ngữ cảnh là thông tin hình học, không phải thông tin về ngoạiại.

Đó là lý do mọi pipeline khớp điểm nghiêm túc đều có *hai tầng*: tầng vẻ ngoài đề xuất ứng viên, tầng hình học kiểm định chúng. Tầng thứ hai không phải một bước làm sạch tùy chọn mà là một phần thiết yếu, và nó là chỗ A/01 quay lại phục vụ chương này.

TRỰC GIÁC

Hình dung bạn phải nối hai bức ảnh chụp cùng một căn phòng từ hai chỗ đứng, bằng cách vẽ các đường nối những điểm tương ứng.

Bạn không bắt đầu bằng cách nhìn từng pixel. Bạn tìm những chỗ *đáng nhớ*: góc bàn, mép tranh, vết ố trên tường. Những chỗ ấy có hai tính chất — chúng khác biệt với xung quanh, và chúng vẫn nhận ra được từ góc nhìn khác. Đó là công việc của detector.

Rồi với mỗi chỗ đáng nhớ, bạn ghi lại nó *trông thế nào* theo một cách không phụ thuộc vào việc bạn đứng đâu — “một góc vuông, sáng ở trên, tối ở dưới”. Đó là descriptor.

Rồi bạn ghép: với mỗi chỗ ở ảnh một, tìm chỗ ở ảnh hai có mô tả giống nhất. Và đây là chỗ bạn sẽ mắc lỗi, vì trong phòng có bốn cái ghế giống hệt nhau.

Cách bạn — một con người — phát hiện lỗi ấy rất đáng chú ý: bạn *không* nhìn kỹ hơn vào cái ghế. Bạn lùi lại và nhìn *tất cả* các đường nối cùng lúc. Nếu hai mươi đường nối đều đi theo một quy luật chung và một đường đi ngược hẳn, bạn biết ngay đường đó sai — không phải vì nó trông sai, mà vì nó *không phù hợp với số đông*.

Đó chính xác là RANSAC, và đó là lý do tăng hình học không thể thiếu. Thông tin để phát hiện khớp sai không nằm trong khớp đó; nó nằm trong quan hệ giữa nó với tất cả các khớp khác.

1.2 Tìm điểm đáng theo dõi

A. Vì sao góc chứ không phải cạnh

Tiêu chí chọn đặc trưng có một cơ sở toán học gọn và đáng tự dẫn một lần.

Xét một cửa sổ nhỏ và hỏi: dịch nó đi một đoạn $\Delta \mathbf{u}$ thì cường độ đổi bao nhiêu? Tổng bình phương chênh lệch, khai triển bậc nhất:

$$E(\Delta \mathbf{u}) = \sum_w [I(\mathbf{u} + \Delta \mathbf{u}) - I(\mathbf{u})]^2 \approx \Delta \mathbf{u}^\top M \Delta \mathbf{u}, \quad M = \sum_w \nabla I \nabla I^\top.$$

Ma trận M cỡ 2×2 là **ma trận cấu trúc**. Hai giá trị riêng của nó phân loại vùng ảnh: cả hai nhỏ nghĩa là vùng phẳng — dịch đi đâu cũng không đổi, không theo dõi được; một lớn một nhỏ nghĩa là cạnh — dịch vuông góc thì đổi, dịch dọc thì không, đây chính là **bài toán cửa sổ**; cả hai lớn nghĩa là góc — dịch theo hướng nào cũng đổi, theo dõi được theo cả hai chiều.

Shi và Tomasi [2] đề nghị dùng trực tiếp $\min(\lambda_1, \lambda_2)$ làm điểm số; Harris dùng một tổ hợp của định thức và vết để tránh tính giá trị riêng. Hai tiêu chí rất gần nhau trong thực hành.

Điều đáng nhận ra, và nó nối chương này với A/04: M chính là *ma trận thông tin của bài toán ước lượng vị trí đặc trưng*. Bài toán “đặc trưng này dịch đi đâu” là một bài toán MAP nhỏ, và M là Hessian của nó. Cạnh không theo dõi được vì M suy biến theo hướng dọc cạnh — đó đúng là hiện tượng ở A/05, gặp ở quy mô một cửa sổ 7×7 pixel. Cùng một nguyên lý xuất hiện ở hai thang bậc khác nhau, và nhận ra điều đó là dấu hiệu đã đọc đúng cả hai chương.

B. Từ cổ điển tới học được

FAST [3] kiểm một vòng tròn 16 pixel quanh ứng viên và hỏi có đủ số pixel liên tiếp sáng hơn hoặc tối hơn tâm không — nhanh tới mức gần như miễn phí, không bắt biến gì cả. **ORB** [4] thêm hướng vào FAST và ghép với descriptor nhị phân BRIEF có xoay; kết quả là một cặp detector-descriptor cực nhanh, khớp bằng khoảng cách Hamming. **SIFT** [1] tìm cực trị trong không gian tỉ lệ và mô tả bằng histogram gradient; chậm hơn nhiều bậc nhưng bất biến tỉ lệ và xoay thật sự.

SuperPoint [5] học cả detector lẫn descriptor bằng tự giám sát. **SuperGlue** [6] thay bước khớp bằng một mạng chú ý nhìn *toàn bộ* hai tập điểm cùng lúc, nên nó khai thác được ngữ cảnh — đây là bước tiến khái niệm quan trọng, vì nó là lần đầu thông tin ngữ cảnh được đưa vào tầng về ngoài thay vì chỉ ở tầng hình học. **LightGlue** [7] làm cùng việc rẻ hơn nhiều với cơ chế thoát sớm. **LoFTR** [8] bỏ hẳn detector và khớp trực tiếp ở mức dày — hoạt động tốt ở vùng ít vân, nơi mọi detector đều thất bại.

CẠM BÃY

Tôi **không** đặt bảng so sánh số liệu giữa các detector ở đây, và lý do thuộc về phương pháp chứ không phải sự lười.

Các con số công bố (số khớp đúng, độ lặp lại, tỉ lệ thành công) phụ thuộc cực mạnh vào bộ dữ liệu, vào ngưỡng, vào cách đếm, và vào việc ai chạy baseline. Hai bài báo báo cáo hai con số cho cùng một cặp phương pháp trên cùng bộ dữ liệu thường *không so sánh được với nhau*, và việc đặt chúng cạnh nhau trong một bảng tạo ra ảo giác so sánh — đúng thứ mà `research-rules.md` mục 6.2 cảnh báo.

Điều *có thể* nói một cách trung thực là các đánh đổi về cơ chế, và đó là nội dung Bảng 1.1. Điều *không thể* nói là “X tốt hơn Y bao nhiêu phần trăm” mà không kèm điều kiện đo đầy đủ.

Cách đúng để chọn cho dự án của bạn không phải đọc bảng của người khác: đó là **đo trên dữ liệu của chính bạn**, trong chính điều kiện triển khai, với chính phần cứng mục tiêu. Chương D/21 lập luận rằng đây không phải lời khuyên cầu toàn mà là yêu cầu bắt buộc, vì khoảng cách giữa EuRoC và một nhà kho thật lớn hơn khoảng cách giữa hai detector bất kỳ.

Bảng 1.1. Các họ front-end theo cơ chế, không theo con số. Cột cuối là tiêu chí chọn thực dụng.

Họ	Cơ chế	Hỏng khi	Chọn khi
FAST + BRIEF/ORB	Kiểm vòng tròn cường độ; descriptor nhị phân	Đổi tỉ lệ mạnh; đổi ánh sáng lớn; vùng ít vân	CPU hạn chế; tốc độ khung cao; chuyển động vừa phải
SIFT và họ	Cực trị không gian tỉ lệ; histogram gradient	Vùng ít vân; đổi về ngoài lớn (mùa, ngày đêm)	Cần bất biến tỉ lệ thật; SfM ngoại tuyến
KLT / optical flow	Theo dõi mảng trực tiếp bằng gradient	Chuyển động lớn; đổi ánh sáng; che khuất	Khung liên tiếp gần nhau; muốn rẻ và chính xác subpixel
Học được (SuperPoint+Li)	Detector và matcher học từ dữ liệu; matcher dùng ngữ cảnh	Ngoài phân bố huấn luyện; cần GPU; khó dự đoán kiểu hỏng	Có GPU; điều kiện khó; cần độ bền với đổi về ngoài
Detector-free (LoFTR, MAST3R)	Khớp dày trực tiếp, không qua điểm	Rất đắt; khó đưa vào vòng lặp thời gian thực	Vùng ít vân; localization ngoại tuyến

Về câu hỏi mở đầu thứ tư — vì sao sản phẩm vẫn dùng ORB — câu trả lời “vì GPU đắt” chỉ là một phần. Ba lý do còn lại đáng giá hơn. *Tính dự đoán được*: ORB hỏng theo những cách đã biết rõ và lặp lại được, còn mạng học được hỏng theo những cách phụ thuộc vào việc dữ liệu hiện tại có giống dữ liệu huấn luyện không — và không ai đo được điều đó tại hiện trường. *Tính tất định*: cùng đầu vào cho cùng đầu ra, bit-exact, điều kiện bắt buộc để debug một lỗi từ hiện trường

(D/20). *Chi phí cập nhật*: đổi một mô hình học được đòi hỏi tái kiểm định toàn bộ, còn đổi một ngưỡng thì không. Ba lý do đó thuộc về D/21 và chúng thường quyết định hơn độ chính xác.

1.3 Khớp: mô tả rời so, hay theo dõi trực tiếp

A. Hai chế độ hoạt động

Có hai cách hoàn toàn khác nhau để tìm tương ứng, và chúng phù hợp với hai tình huống khác nhau — nhằm chúng là một lỗi thiết kế phổ biến.

Khớp bằng descriptor không giả định gì về vị trí: trích đặc trưng ở cả hai ảnh, mô tả chúng, rồi tìm ứng viên gần nhất trong không gian descriptor. Dùng được cho hai ảnh *bất kỳ*, kể cả cách nhau hàng giờ hoặc chụp từ hai thiết bị khác nhau. Chi phí: trích đặc trưng ở cả hai ảnh, và tìm kiếm trong không gian nhiều chiều.

Theo dõi bằng optical flow giả định chuyển động nhỏ: lấy vị trí đặc trưng ở khung trước làm khởi đầu, rồi giải cho độ dịch làm cường độ khớp nhất. KLT làm điều này bằng Gauss-Newton trên một mảng nhỏ, với tháp ảnh để chịu được chuyển động lớn hơn [9]. Rẻ hơn nhiều vì không phải trích và mô tả lại; chính xác subpixel tốt hơn vì nó tối ưu trực tiếp trên cường độ. Hồng khi chuyển động quá lớn hoặc ánh sáng đổi.

Quy tắc chọn gọn: *theo dõi giữa các khung liên tiếp, khớp descriptor khi cần nối hai khung xa nhau*. Đây là lý do VINS-Mono dùng KLT cho tracking [10] còn ORB-SLAM dùng descriptor — hai lựa chọn kiến trúc khác nhau phù hợp với hai ưu tiên khác nhau, không phải một cái đúng một cái sai. Nhưng mọi hệ thống đều cần descriptor ở đâu đó, vì loop closure (B/11) và relocalization (B/12) bản chất là khớp giữa hai khung rất xa nhau về thời gian.

Baker và Matthews [11] chỉ ra một biến thể quan trọng: *inverse compositional*, trong đó Hessian được tính *một lần* trên mảng tham chiếu thay vì tính lại mỗi vòng lặp. Cùng kết quả, chi phí thấp hơn nhiều bậc. Đây là loại tối ưu hoá đáng biết vì nó không đánh đổi gì cả.

B. Ratio test và giới hạn của nó

Sau khi tìm ứng viên gần nhất, câu hỏi là có chấp nhận không. Dùng ngưỡng tuyệt đối trên khoảng cách descriptor hoạt động kém, vì thang khoảng cách khác nhau giữa các loại đặc trưng và giữa các vùng ảnh.

Ratio test của Lowe so khoảng cách tới ứng viên gần nhất với ứng viên gần *thứ hai*, và chỉ chấp nhận nếu tỉ số đủ nhỏ. Trực giác: nếu đặc trưng thật sự đặc biệt thì ứng viên đúng phải gần hơn hẳn mọi ứng viên khác; nếu có hai ứng viên gần ngang nhau thì ta không có cơ sở chọn. Đây là một phép thử *tương đối*, nên nó tự thích ứng với thang khoảng cách địa phương — và đó là lý do nó bền hơn ngưỡng tuyệt đối rất nhiều.

Nhưng nó có một giới hạn quan trọng, và đó là câu trả lời cho câu hỏi mở đầu thứ năm. Ratio test loại được trường hợp *có nhiều ứng viên giống nhau trong tầm nhìn*. Nó không loại được trường hợp cánh cửa thứ hai không nằm trong tập ứng viên — chẳng hạn vì nó ở ngoài khung hình hiện tại, hoặc vì ta chỉ tìm kiếm trong một vùng nhỏ. Lúc đó ứng viên sai là ứng viên duy nhất, tỉ số rất tốt, và khớp được chấp nhận với độ tin cậy cao.

Thứ cứu được chỉ có thể là hình học: kiểm định rằng khớp này nhất quán với đa số các khớp khác (mục 5), hoặc nhất quán với một tiên nghiệm về chuyển động. Đây lại là bài học ở khối *Trực giác*: thông tin để phát hiện khớp sai không nằm trong khớp đó.

1.4 Hai thứ bị coi nhẹ nhất

A. Phân bố không gian

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là một trong những điểm có tỉ lệ giá trị trên độ phức tạp cao nhất trong cả cây.

Cách cài đặt ngây thơ: tính điểm số cho mọi ứng viên, sắp xếp, lấy N cái tốt nhất. Kết quả gần như luôn là các đặc trưng *tụ lại* ở vùng có vân mạnh — một tấm poster trên tường, một tán lá — và cả vùng còn lại của ảnh không có đặc trưng nào.

Vì sao điều đó tệ? Quay lại Jacobian ở A/03 công thức (??): đóng góp của một quan sát vào ma trận thông tin phụ thuộc vào *vị trí của nó trong ảnh* qua các số hạng X/Z và Y/Z . Các đặc trưng tụ ở một chỗ đóng góp các hàng Jacobian gần như *tuyến tính phụ thuộc* nhau, nên chúng thêm rất ít thông tin mới. Hệ quả là Λ có số điều kiện lớn, và bài toán gần suy biến theo một hướng — đúng loại suy biến số học ở A/05 mục 2C. Một trăm đặc trưng trải đều cho nhiều thông tin hơn năm trăm đặc trưng tụ một chỗ.

Có một lý do thứ hai, độc lập và cũng quan trọng: *độ bền*. Nếu mọi đặc trưng ở trên một vật, và vật đó bị che hoặc bắt đầu di chuyển, hệ thống mất tất cả cùng lúc. Phân bố đều là một dạng đa dạng hoá rủi ro.

Ba cách cài đặt, theo thứ tự tinh vi. *Lưới*: chia ảnh thành ô, giữ tối đa k đặc trưng tốt nhất mỗi ô. Rẻ, hiệu quả, và đủ cho hầu hết trường hợp. *Quadtree*: chia thích ứng, tinh hơn ở vùng đông — ORB-SLAM dùng cách này [12]. *ANMS* (adaptive non-maximal suppression): giữ đặc trưng theo bán kính triệt tiêu giảm dần, cho phân bố đều nhất nhưng đắt nhất.

B. Độ chính xác subpixel

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và nó là điểm nối trực tiếp nhất giữa front-end và A/06.

Vị trí đặc trưng ở độ phân giải pixel nguyên cho $\sigma \approx 0,5$ px (sai số lượng tử hoá của một biến phân bố đều trên một pixel là $1/\sqrt{12} \approx 0,29$ px, nhưng trong thực tế sai số lớn hơn thế vì còn nhiễu và lượng tử hoá không hoàn hảo). Tinh chỉnh subpixel — khớp một parabol vào hàm điểm số quanh cực đại, hoặc chạy vài vòng KLT — giảm con số đó xuống mức thường được nêu là 0,1 tới 0,2 px.

CẠM BẤY

Hai con số vừa nêu cần một cảnh báo. Khoảng 0,1–0,2 px là con số **thường được nêu trong cộng đồng** cho tinh chỉnh subpixel trên ảnh có vân tốt, nhưng **tôi không dẫn được một nguồn đo có điều kiện đầy đủ cho nó**, và nó phụ thuộc mạnh vào độ tương phản, mức nhiễu cảm biến, và độ sắc nét của đặc trưng.

Theo `learning-rules.md` mục 5, một con số không có điều kiện đo là một con số vô nghĩa, nên cách đúng là: **tự đo σ của chính front-end của bạn**, trên chính dữ liệu của bạn. Cách đo rẻ nhất: chụp một cảnh tĩnh với camera cố định trên chân máy, theo dõi đặc trưng qua vài trăm khung, và tính độ lệch chuẩn của vị trí từng đặc trưng. Con số thu được là σ *thật* của bạn, đã bao gồm cả nhiễu cảm biến lẫn giới hạn thuật toán.

Con số đó đáng đo vì nó là **trọng số $\Omega = \sigma^{-2}$** cho mọi phần dư chiếu ở back-end. Đặt sai nó thì mọi hiệp phương sai đầu ra sai theo, và bạn rơi vào đúng nguồn số hai của bệnh quá tự tin ở A/06 mục 4B. Đây là một trong những phép đo rẻ nhất và bị bỏ qua nhiều nhất trong cả quy trình.

Vì sao σ quan trọng tới vậy? Ba đường. *Một*: nó vào thẳng Ω_k trong hệ chuẩn (??), nên nó quyết định trọng số tương đối giữa nhân tử thị giác và mọi nhân tử khác — IMU, odometry, GNSS. *Hai*: hiệp phương sai đầu ra tỉ lệ với σ^2 , nên giảm σ ba lần thì hiệp phương sai giảm chín lần; đó là một cải thiện lớn hơn hầu hết các thay đổi thuật toán. *Ba*: qua σ_d trong công thức (??), nó quyết định trực tiếp tầm hữu ích của stereo. Với $b = 12$ cm, $f = 400$ px, ở $z = 5$ m: $\sigma = 0,5$ px cho $\sigma_z \approx 26$ cm, còn $\sigma = 0,15$ px cho $\sigma_z \approx 8$ cm. Cùng phần cứng, ba lần khác biệt về chất lượng độ sâu.

1.5 Loại bỏ outlier

A. RANSAC và vì sao cỡ mẫu quan trọng

RANSAC [13] lặp bốn bước: lấy ngẫu nhiên s tương ứng (cỡ mẫu tối thiểu của mô hình), khớp mô hình, đếm inlier, giữ mô hình nhiều inlier nhất.

Số vòng lặp cần để có xác suất p lấy được ít nhất một mẫu toàn inlier, với tỉ lệ inlier w :

$$N = \frac{\log(1-p)}{\log(1-w^s)}. \quad (1.1)$$

Suy dẫn: xác suất một mẫu toàn inlier là w^s ; xác suất N mẫu đều hỏng là $(1-w^s)^N$; đặt bằng $1-p$ và lấy log. (Cửa a.)

Ví dụ số trả lời câu hỏi mở đầu thứ nhất. Với $w = 0,5$ và $p = 0,99$:

Thuật toán năm điểm ($s = 5$): $w^s = 0,5^5 = 0,03125$, nên $N = \log(0,01)/\log(0,96875) \approx -4,605/(-0,03175) \approx 146$ vòng.

Thuật toán tám điểm ($s = 8$): $w^s = 0,5^8 = 0,00391$, nên $N \approx -4,605/(-0,003914) \approx 1177$ vòng.

Tám lần khác biệt cho ba điểm mẫu thêm. (Cửa kiểm chứng b, tính tay.) Và với $w = 0,3$, chênh lệch thành 170 so với 6300 vòng — gần bốn mươi lần. Đây là lý do thực dụng chính để dùng thuật toán năm điểm, và nó lớn hơn nhiều so với ấn tượng “chỉ bớt ba điểm”. Quan hệ là hàm mũ theo s , không tuyến tính.

B. Cải tiến và giới hạn

PROSAC [14] lấy mẫu theo thứ tự chất lượng khớp thay vì ngẫu nhiên đều — vì khớp có điểm số tốt có xác suất là inlier cao hơn. Hội tụ nhanh hơn nhiều khi thang điểm có ý nghĩa. **MAGSAC++** [15] bỏ ngưỡng inlier cứng bằng cách tích phân trên các mức ngưỡng — giải quyết một vấn đề thực tế đáng kể, là ngưỡng inlier thường được chọn tùy tiện và kết quả nhạy với nó. **LO-RANSAC** chạy một bước tinh chỉnh cục bộ trên tập inlier mỗi khi tìm được mô hình tốt hơn, cải thiện rõ với chi phí nhỏ.

Ba giới hạn của RANSAC đáng nhớ vì chúng là nơi nó thất bại một cách im lặng.

Một: nó tìm *tập nhất quán lớn nhất*, không phải nghiệm đúng. Nếu outlier có cấu trúc — tất cả nằm trên một vật đang di chuyển — thì chúng nhất quán với nhau, và nếu vật đó chiếm đa số khung hình, RANSAC sẽ trung thành chọn vật động làm inlier. Kết quả sai nhưng nhất quán, tức im lặng. Xử lý ở D/19.

Hai: nó cần một mô hình. Trong cấu hình suy biến ở A/01 mục 6, mô hình không xác định duy nhất, nên RANSAC báo nhiều inlier cho một mô hình vô nghĩa. Đây chính là lý do khối *Cắm bẫy* ở chương đó cấm dùng số inlier làm thước đo chất lượng.

Ba: nghiệm của nó chỉ để khởi tạo. Mô hình khớp từ mẫu tối thiểu không dùng *tất cả* inlier, nên nó không tối ưu. Luôn phải theo sau bởi một bước tinh chỉnh trên toàn bộ inlier — và bước đó chính là bài toán MAP nhỏ của A/04.

C. Tiên nghiệm chuyển động: cách rẻ nhất cải thiện front-end

Một kỹ thuật đơn giản mà hiệu quả bất ngờ, và nó là một trong những chỗ mà kiến trúc hệ thống trả công.

Nếu có tiên nghiệm về chuyển động giữa hai khung — từ IMU, từ odometry bánh xe, hoặc chỉ từ mô hình vận tốc không đổi — ta dự đoán được mỗi landmark sẽ chiếu về *đâu* trong khung mới. Thay vì tìm kiếm toàn ảnh, ta tìm trong một vùng nhỏ quanh vị trí dự đoán.

Ba lợi ích cùng lúc, và cả ba đều lớn. *Rẻ hơn*: vùng tìm kiếm nhỏ hơn hàng chục lần. *Ít khớp sai hơn*: bớt ứng viên nghĩa là bớt cơ hội nhầm — và điều này tấn công trực tiếp vào perceptual aliasing, vì cánh cửa thứ hai thường nằm ngoài vùng tìm kiếm. *Tỉ lệ inlier cao hơn*: w tăng, nên theo (1.1) số vòng RANSAC giảm theo hàm mũ.

Đây là một ví dụ rõ ràng của việc hợp nhất cảm biến giúp *front-end* chứ không chỉ back-end — một điểm hay bị bỏ qua khi người ta nghĩ về IMU chỉ như một nguồn ràng buộc bổ sung trong đồ thị. Chi tiết ở C/15 và ../IMU_Fusion/.

Kiến trúc này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [16]	ORB + quadtree phân bố; tìm kiếm có hướng dẫn bằng mô hình chuyển động; RANSAC PnP	Mục 4A và 5C trong một hệ thống hoàn chỉnh
VINS-Mono [10]	KLT tracking + lưới phân bố; IMU dự đoán vùng tìm kiếm	Lựa chọn kiến trúc đối lập với ORB-SLAM — xem mục 3A
COLMAP [17]	SIFT + ratio test + kiểm định hình học đa mô hình	Pipeline hai tầng đầy đủ nhất trong phần mềm mã nguồn mở
HLoc [18]	SuperPoint + SuperGlue/LightGlue	Front-end học được trong pipeline localization thật
SVO-Pro [19]	Trích đặc trưng thưa + căn chỉnh direct; tinh chỉnh subpixel bằng tối ưu cường độ	Mình hoạ mục 4B: độ chính xác subpixel đến từ tối ưu trực tiếp trên cường độ

MINI PROJECT SLAM · Đo σ thật của front-end của bạn, rồi đo ảnh hưởng của phân bố đặc trưng · 3 ngày

Mục tiêu	biến hai con số mà mọi người đoán — σ subpixel và ảnh hưởng của phân bố — thành hai con số bạn đo được trên phần cứng của mình.
Dữ liệu	camera của bạn trên chân máy cố định, chụp một cảnh tĩnh có vân đa dạng (vài trăm khung); cộng thêm một chuỗi EuRoC để có chân trị.
Các bước	<i>Phần một</i> : theo dõi đặc trưng qua toàn bộ chuỗi tĩnh, tính độ lệch chuẩn vị trí của từng đặc trưng, vẽ histogram — đó là σ thật của bạn; lặp lại có và không có tinh chỉnh subpixel, và với đặc trưng có độ tương phản cao và thấp riêng. <i>Phần hai</i> : trên chuỗi EuRoC, chạy BA với hai chiến lược chọn đặc trưng — “top-N theo điểm số” và “lưới phân bố” — với <i>cùng tổng số</i> đặc trưng; đo số điều kiện của Λ và ATE cho cả hai.
Xong khi	bạn có một con số σ của riêng mình kèm điều kiện đo, và bạn dùng nó làm trọng số trong mọi thứ về sau thay vì một hằng số đoán. Và bạn có hai đường cong số điều kiện cho thấy phân bố lưới cho Λ tốt hơn đáng kể ở cùng số đặc trưng — nếu không quan sát được, hãy kiểm xem cảnh của bạn có vân quá đều không, và đó cũng là một kết quả đáng ghi.

Nối với

A/06-bat-dinh-va-lan-truyen-sai-so dùng σ này làm đầu vào cho toàn bộ ngân sách sai số; C/18-danh-gia-va-phuong-phap-luan biến phần hai thành một ablation study có kỷ luật.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Từ $N = \log(1 - p) / \log(1 - w^s)$ với $w = 0,5$, $p = 0,99$: năm điểm cần khoảng 146 vòng, tám điểm cần khoảng 1177 — *tám lần*. Với $w = 0,3$ thì chênh gần bốn mươi lần. Lớn hơn trực giác vì quan hệ là *hàm mũ* theo s : mỗi điểm thêm vào nhân xác suất thành công của một mẫu với w , nên với $w = 0,5$ thì ba điểm thêm chia xác suất cho tám. Đây là lý do thực dụng chính để dùng thuật toán năm điểm, cùng với lý do quan trọng không kém là nó không suy biến với cảnh phẳng (A/01 mục 6B).
- A2. *Phân bố không gian* của đặc trưng. Hệ tụ đặc trưng ở vùng vãn mạnh cho các hàng Jacobian gần tuyến tính phụ thuộc — theo công thức Jacobian ở A/03, đóng góp phụ thuộc vị trí trong ảnh — nên Λ có số điều kiện lớn và bài toán gần suy biến. Một trăm đặc trưng trải đều cho nhiều thông tin hơn năm trăm đặc trưng tụ một chỗ. Lý do thứ hai, độc lập: nếu mọi đặc trưng ở trên một vật thì khi vật bị che hoặc di chuyển, hệ mất tất cả cùng lúc. Cách chữa rẻ nhất là lưới; tinh hơn thì quadtree hoặc ANMS.
- A3. Vì σ là trọng số $\Omega = \sigma^{-2}$ cho mọi phần dư chiều, nên nó quyết định trọng số *tương đối* giữa nhân tử thị giác và mọi nhân tử khác trong đồ thị. Hiệp phương sai đầu ra tỉ lệ σ^2 , nên giảm σ hơn ba lần thì hiệp phương sai giảm hơn mười lần — lớn hơn hầu hết cải tiến thuật toán. Và qua σ_d trong $\sigma_z = z^2 \sigma_d / (bf)$, nó quyết định tầm hữu ích của stereo: ở $z = 5$ m với $b = 12$ cm, $f = 400$ px, $\sigma = 0,5$ px cho $\sigma_z \approx 26$ cm còn $\sigma = 0,15$ px cho ≈ 8 cm. Cùng phần cứng, ba lần khác biệt.
- A4. “GPU đắt” chỉ là một phần. Ba lý do quan trọng hơn, và cả ba thuộc về D/21. *Tính dự đoán được*: ORB hỏng theo cách đã biết và lặp lại được; mạng học được hỏng khi dữ liệu lệch khỏi phân bố huấn luyện, và không ai đo được điều đó tại hiện trường. *Tính tất định*: cùng đầu vào cho cùng đầu ra bit-exact, điều kiện bắt buộc để tái hiện một lỗi từ hiện trường. *Chi phí cập nhật*: đổi mô hình đòi tái kiểm định toàn bộ; đổi ngưỡng thì không. Trong sản phẩm, ba yếu tố đó thường quyết định hơn độ chính xác trung bình.
- A5. Vì ratio test so ứng viên gần nhất với ứng viên gần *thứ hai trong tập ứng viên*. Nếu cánh cửa thứ hai không nằm trong tập — ngoài khung hình, hoặc ngoài vùng tìm kiếm — thì ứng viên sai là ứng viên duy nhất, tỉ số rất tốt, và khớp được chấp nhận với độ tin cậy cao. Thứ cứu được chỉ có thể là *hình học*: kiểm định rằng khớp này nhất quán với đa số khớp khác (RANSAC), hoặc nhất quán với tiên nghiệm chuyển động. Nguyên tắc chung: thông tin để phát hiện một khớp sai không nằm trong khớp đó mà nằm trong quan hệ của nó với các khớp khác — và ở quy mô lớn hơn, đây chính là vấn đề perceptual aliasing ở B/11.

NÓI LẠI THẬT ĐƠN GIẢN

Đưa cho một người hai bức ảnh chụp cùng một căn phòng từ hai chỗ đứng và bảo họ nối các điểm tương ứng. Họ sẽ làm ba việc, theo đúng thứ tự.

Đầu tiên họ tìm những chỗ *đáng nhớ*. Không phải giữa bức tường trắng — chỗ đó nhìn đâu cũng như nhau. Họ tìm góc bàn, mép khung tranh, vết bẩn trên sàn. Nguyên tắc: một chỗ đáng nhớ là chỗ mà dịch đi một chút theo *bất kỳ* hướng nào thì nó trông khác ngay. Một đoạn mép bàn thẳng thì không đạt — trượt dọc theo mép, nó vẫn y như cũ.

Thứ hai họ ghép. Với mỗi chỗ đáng nhớ ở ảnh một, tìm chỗ giống nhất ở ảnh hai. Và ở bước này họ chắc chắn sẽ mắc lỗi, vì trong phòng có bốn cái ghế giống hệt nhau.

Thứ ba — và đây là bước quan trọng nhất — họ *lùi lại nhìn tất cả các đường nối cùng lúc*. Nếu hai mươi đường đi theo một quy luật chung và một đường đi ngược hẳn, họ gạch đường đó đi. Không phải vì nhìn kỹ hơn vào cái ghế, mà vì đường đó không hợp với số đông.

Đó là toàn bộ chương này, và bài học lớn nhất nằm ở bước ba: *không có cách nào nhìn kỹ hơn vào một cái ghế để biết nó là cái nào*. Thông tin đó không tồn tại trong cái ghế. Nó chỉ tồn tại trong việc cái ghế ấy ở đâu so với mọi thứ khác.

Còn hai chi tiết nhỏ mà ít ai để ý nhưng lại quan trọng hơn vẻ ngoài. Một: đừng chọn tất cả các chỗ đáng nhớ ở cùng một góc ảnh — trải chúng ra khắp bức ảnh, vì mỗi vùng nói cho bạn một chuyện khác nhau. Hai: hãy xác định vị trí của chúng *chính xác nhất có thể*, chính xác hơn cả một ô pixel. Nghe như chuyện vụn vặt, nhưng con số ấy nhân lên qua toàn bộ phép tính phía sau, và cải thiện nó ba lần thì kết quả cuối cải thiện gấp mười.

Chỗ không rút gọn được: chỗ không rút gọn được là những căn phòng mà *mọi thứ* đều lặp lại — một hành lang kho hàng dài với các kệ giống hệt nhau. Ở đó bước ba cũng thất bại, vì số đông cũng có thể sai một cách nhất quán. Đó là bài toán khó nhất trong triển khai thật, và B/11 nói rằng nó vẫn chưa có lời giải tốt.

Một câu: Không có gì trong một khớp cho biết nó đúng hay sai — chỉ có quan hệ của nó với mọi khớp khác mới cho biết.

1.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *không descriptor nào phân biệt được hai vật thật sự giống hệt nhau, nên tăng hình học là bắt buộc* — gần như đúng theo định nghĩa nếu “giống hệt” hiểu chặt. Nó trở nên có thể bác bỏ khi ta hỏi về trường hợp thực tế: hai cánh cửa gần giống nhau. Ở đó, một descriptor đủ tốt có thể khai thác các khác biệt nhỏ — vết xước, khác biệt ánh sáng — và mệnh đề bị bác bỏ nếu một hệ thuần về ngoài đạt tỉ lệ khớp sai thấp ngang hệ có kiểm định hình học trên môi trường lặp lại. Tôi cho là không, nhưng LoFTR và MAST3R đi theo hướng khai thác ngữ cảnh rộng hơn, nên ranh giới đang dịch chuyển và tôi không chắc nó dừng ở đâu (*tôi suy ra, chưa đối chiếu nguồn*).

Mệnh đề thứ hai — *phân bố không gian ảnh hưởng tới số điều kiện của Λ* — suy ra trực tiếp từ cấu trúc Jacobian (của a) và kiểm được bằng mini project (của b). Nó bị bác bỏ nếu thí nghiệm cho thấy số điều kiện không khác nhau đáng kể giữa hai chiến lược chọn ở cùng số đặc trưng. Một trường hợp mà nó *thật sự* không đúng: khi cảnh có vân đều khắp nơi, top-N tự nhiên đã trải đều, và hai chiến lược trùng nhau.

Mệnh đề thứ ba — *tính chỉnh subpixel cho σ khoảng 0,1–0,2 px* — là con số tôi không có nguồn đo, và tôi đã nói rõ điều đó ở khối *Cạm bẫy*. Nó dễ bác bỏ nhất trong chương: chỉ cần đo, và mini project chính là phép đo. Nếu σ thật của bạn là 0,4 px thì mọi tính toán ngân sách sai số dựa trên 0,15 px đều sai gấp bảy lần về phương sai.

Một mệnh đề mềm: chương này khẳng định *front-end quyết định kiểu hồng, back-end quyết định độ chính xác*. Điều đó bị bác bỏ nếu trong một khảo sát ca hồng thật, phần lớn thất bại bắt nguồn từ back-end — phân kỳ số học, cực tiểu địa phương — chứ không từ data association. Tôi không có số liệu và đây là chỗ tôi phát biểu dựa trên cấu trúc của vấn đề (*tôi suy ra, chưa đối chiếu nguồn*); D/21 đề nghị cách thu số liệu đó.

1.7 Kết nối

Nói lên trên. A/01-hình-hoc-da-thi cung cấp tầng kiểm định hình học ở mục 5, và đồng thời nhận từ chương này đầu vào của nó — hai chương nối vòng với nhau. A/02-mo-hình-camera là bước gỡ méo phải chạy trước khi dùng bất kỳ ràng buộc hình học nào. A/03-lie-group-va-toi-uu-tren-da-tap giải thích vì sao phân bố đặc trưng ảnh hưởng tới điều kiện số: qua cấu trúc của Jacobian. A/05-observability-gauge-va-suy-bien là chỗ ma trận cấu trúc M ở mục 2A được nhìn đúng bản chất — một ma trận thông tin suy biến ở quy mô nhỏ. A/06-bat-dinh-va-lan-truyen-sai-so là chương nhận σ từ mục 4B và biến nó thành ngân sách sai số.

Nổi xuống dưới. B/08-mo-hinh-phan-du là lựa chọn tiếp theo: một khi có tương ứng, phần dư viết thế nào — và với direct method thì chương này gần như biến mất, vì không còn bước trích đặc trưng. B/09-paradigm-uoc-luong-back-end nhận đầu ra của chương này làm đầu vào, và giả định rằng outlier đã được loại — giả định mà mục 5B cảnh báo là không bao giờ đúng hoàn toàn. B/11-loop-closure-va-nhat-quan-toan-cuc là chương mà bài toán của chương này xuất hiện lại ở quy mô lớn hơn và với hậu quả nghiêm trọng hơn: khớp sai giữa hai khung liên kế chỉ làm hỏng một quan sát, còn khớp sai giữa hai địa điểm làm hỏng cả bản đồ. B/12-relocalization cũng vậy.

Nổi xa hơn. C/15-hop-nhat-da-cam-bien là chỗ mục 5C mở rộng: tiên nghiệm chuyển động từ bất kỳ cảm biến nào đều cải thiện front-end theo ba đường cùng lúc. C/17-hoc-may-trong-slam bàn kỹ hơn về front-end học được và lập luận rằng đây là chỗ học máy đóng góp rõ ràng nhất cho SLAM. D/19-ben-vung-trong-the-gioi-thuc xử lý ba giới hạn của RANSAC ở mục 5B, đặc biệt là trường hợp vật động chiếm đa số. D/20-toi-uu-hieu-nang-va-he-thong là chỗ chi phí của front-end trở thành ràng buộc cứng — nó thường chiếm phần lớn ngân sách tính toán của cả hệ.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao cạnh không theo dõi được còn góc thì được? Trả lời bằng ma trận cấu trúc, rồi nói nó liên hệ thế nào với A/05.
2. Tự dẫn lại công thức số vòng RANSAC, rồi tính cho $s = 4$ (homography) với $w = 0,4$, $p = 0,99$.
3. Nêu hai chế độ khớp — descriptor và optical flow — và nói mỗi cái phù hợp với tình huống nào và vì sao.
4. Ratio test cứu được trường hợp nào và không cứu được trường hợp nào? Nêu cơ chế duy nhất cứu được trường hợp thứ hai.
5. Vì sao phân bố đặc trưng ảnh hưởng tới độ chính xác nhiều hơn số lượng đặc trưng? Nêu hai lý do độc lập.
6. σ subpixel của bạn là bao nhiêu, bạn đo nó thế nào, và nó ảnh hưởng tới ba thứ nào ở back-end?
7. Nêu ba giới hạn của RANSAC, và với mỗi cái nêu một tình huống thực tế mà nó thất bại im lặng.

Nguồn

- [1] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004, mức tin A.
- [2] J. Shi and C. Tomasi, “Good Features to Track,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 1994.
- [3] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, mức tin A, 2006.
- [4] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2011.
- [5] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [6] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [7] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [8] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [9] J.-Y. Bouguet, “Pyramidal Implementation of the Affine Lucas Kanade Feature Tracker: Description of the Algorithm,” Intel Corporation, tech. rep., 2001, mức tin B; báo cáo kỹ thuật, là tài liệu tham chiếu de-facto cho KLT thấp.
- [10] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [11] S. Baker and I. Matthews, “Lucas-Kanade 20 Years On: A Unifying Framework,” *International Journal of Computer Vision*, vol. 56, no. 3, pp. 221–255, 2004, mức tin A; nguồn của inverse compositional.
- [12] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [13] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [14] O. Chum and J. Matas, “Matching with PROSAC — Progressive Sample Consensus,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2005.
- [15] D. Barath, J. Noskova, M. Ivashechkin, et al., “MAGSAC++, a Fast, Reliable and Accurate Robust Estimator,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2020.
- [16] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.

- [17] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [18] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2019.
- [19] C. Forster, Z. Zhang, M. Gassner, et al., "SVO: Semidirect Visual Odometry for Monocular and Multicamera Systems," *IEEE Transactions on Robotics*, vol. 33, no. 2, pp. 249–265, 2017, mức tin A.